



**Universidad
Indoamérica**

Manual Técnico

Asistente Virtual Universitario Interactivo (ARIA)

Desarrollador: Emilio Espinoza

15 de septiembre de 2026

Índice

1. Introducción y Resumen Arquitectónico	3
2. Arquitectura del Frontend	3
3. Arquitectura del Backend (FastAPI)	3
3.1. Módulo de Visión Artificial	3
3.2. Pipeline de IA (LLMs y TTS)	3
4. El Motor RAG (Retrieval-Augmented Generation)	4
4.1. Indexación y Embeddings	4
4.2. Retrieval y Generación Aumentada	5

1. Introducción y Resumen Arquitectónico

Este manual detalla el diseño, arquitectura y componentes de software del Asistente Virtual (Aria). El proyecto sigue un patrón estructurado de **Cliente-Servidor**, combinando un motor 3D renderizado en tiempo real en el frontend con una robusta arquitectura RAG (Retrieval-Augmented Generation) en el backend.

2. Arquitectura del Frontend

El cliente web está desarrollado utilizando HTML5, CSS Vanilla y JavaScript (ES6+). Sus pilares tecnológicos son:

- **Motor 3D (Three.js):** Se encarga del renderizado espacial y la manipulación de la cámara.
- **Formato VRM:** El avatar está basado en el estándar japonés `.vrm`. A través de la librería `@pixiv/three-vrm`, se manipulan huesos, mallas y blendshapes (expresiones faciales).
- **Sincronización Labial (Lip-sync):** El sistema crea un analizador de frecuencias con la Web Audio API (`AudioContext.createAnalyser()`). En cada frame, extrae la amplitud del audio proveniente del backend (síntesis de voz) y ajusta dinámicamente el hueso de la mandíbula del modelo 3D.
- **Renderizado Markdown:** Emplea `marked.js` para parsear las respuestas crudas del LLM en formato HTML visual (tablas, negritas, listas).

3. Arquitectura del Backend (FastAPI)

El servidor central está escrito en Python 3 utilizando **FastAPI**. Actúa como orquestador de todos los micro-módulos.

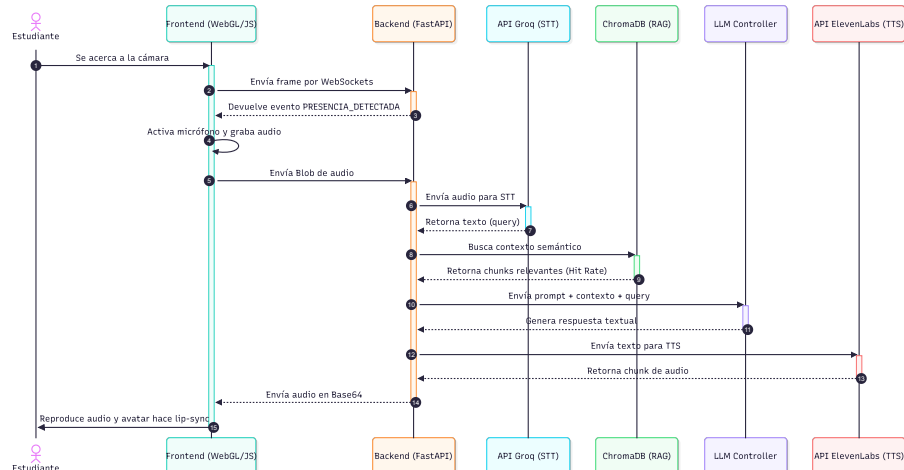
3.1. Módulo de Visión Artificial

Para crear inmersión, el backend procesa imágenes utilizando **OpenCV**.

1. El frontend envía fotogramas capturados por la webcam vía **WebSockets**.
2. El backend aplica un algoritmo clasificador de *Haar Cascades* para detección de rostros frontales (`haarcascade_frontalface_default.xml`).
3. Si se detecta un usuario constante frente a la pantalla, se activa el agente y este inicia la interacción conversacional.

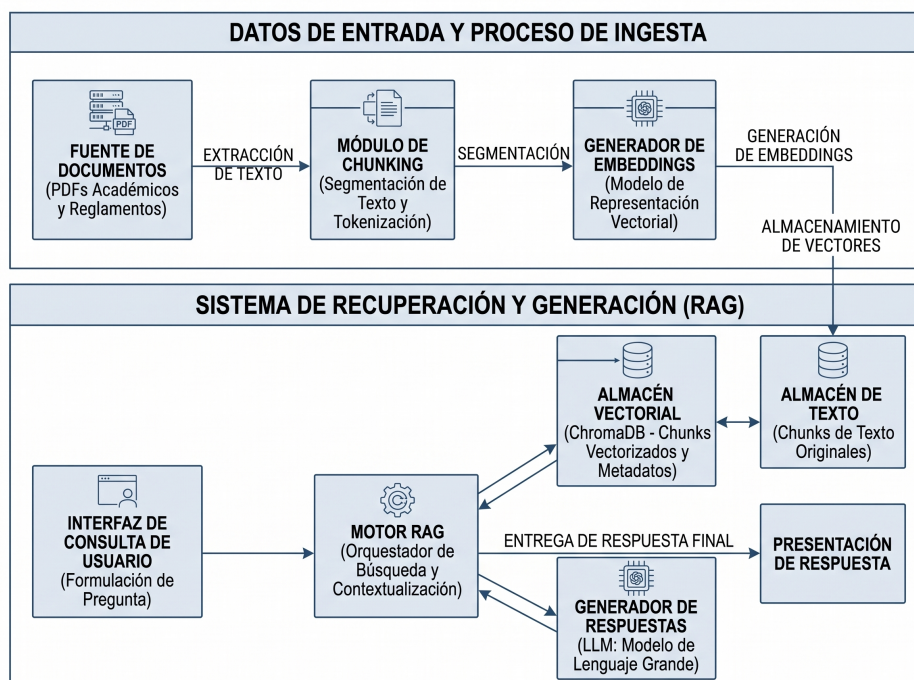
3.2. Pipeline de IA (LLMs y TTS)

El backend abstrae la conexión a múltiples modelos de lenguaje, permitiendo una fácil transición entre APIs de proveedores como NVIDIA (NIM), Groq (Llama 3), Google Gemini o despliegues privados usando OpenWebUI (Qwen local). La síntesis de voz (TTS) se ejecuta a través de ElevenLabs utilizando un modelo optimizado (`eleven_multilingual_v2`), lo que garantiza voces orgánicas en milisegundos y se envían como flujos binarios binarios (Blob/AudioBuffer) hacia el frontend.



4. El Motor RAG (Retrieval-Augmented Generation)

El componente más crítico del sistema cognitivo es su módulo RAG. Dado que los modelos de lenguaje genéricos carecen de conocimiento específico sobre la malla y reglamentos de la Universidad Indoamérica, el motor RAG funciona como una memoria institucional.



4.1. Indexación y Embeddings

- El pipeline utiliza `pymupdf411m` para convertir archivos PDF altamente estructurados en formato Markdown, conservando la integridad matemática de las tablas (por ejemplo, las mallas curriculares de TI).
- El texto se fragmenta mediante el `MarkdownTextSplitter`.

- Cada fragmento es vectorizado utilizando un modelo Transformer de HuggingFace (**all-MiniLM-L6-v2**) que lo mapea a un espacio semántico de 384 dimensiones.
- Estos vectores se almacenan en memoria utilizando la base de datos de grafos/vectores **ChromaDB**.

4.2. Retrieval y Generación Aumentada

Cuando el usuario envía un "prompt":

1. El sistema vectoriza la petición original.
2. Realiza una búsqueda por *similitud de cosenos* en ChromaDB para extraer los fragmentos documentales más relevantes.
3. Estos fragmentos se concatenan a un *System Prompt* restrictivo (*.Eres Aria... Responde únicamente basado en los siguientes fragmentos*").
4. El LLM evalúa la petición junto al contexto local, generando una respuesta de altísima precisión libre de alucinaciones.